

Herald

모든 알림은 명령어 구문 없이 정확한 목적지에 도달합니다.

요약

Herald는 시스템 이벤트를 수집하여 여러 알림 채널로 안정적으로 전송해 모든 알림이 제때 정확한 곳에 도달하도록 합니다. 구독자는 자연스러운 일상 언어로 소통하며, Herald는 세 단계의 프로세스(명령어 빠른 경로 → LLM 의도 추론 → 명확화 후속 조치)를 통해 사용자의 의도를 파악합니다.

간략 설명

이벤트 수집 및 다중 채널 알림 전송 시스템입니다. Herald는 시스템 이벤트를 메신저 채널을 통해 정확한 목적지로 안정적으로 전달하며, 구독자가 자연스러운 언어로 소통할 수 있도록 지원합니다. 이를 위해 명령어 빠른 경로, LLM 기반 의도 추론, 그리고 명확화 요청 후속 조치라는 세 가지 방식을 활용합니다.

상세 설명

Herald는 시스템 이벤트가 실제로 대응 가능한 사람에게 전달되도록 보장하는 알림 인프라입니다. 대부분의 자체 개발 알림 시스템이 조용히 실패하는 지루하지만 중요한 영역을 담당하죠. 이 시스템은 이벤트를 수집해 여러 알림 채널로 안정적으로 전송하며, 알림이 유실되거나 잘못된 채널로 전송되거나 소음 속에 파묻혀 대응 시기를 놓치는 일반적인 실패 사례를 차단합니다. 하지만 안정적인 전달은 절반에 불과합니다. 나머지 절반은 사람이 알림에 응답하려 할 때 벌어지는 일입니다. Herald는 사용자가 알림 봇과 상호작용하기 위해 엄격한 명령어 구문을 암기해야 하는 기존 방식을 거부합니다. 구독자는 그저 자연스러운 언어로 메시지를 작성하면 되고, Herald는 세 단계의 체계적인 프로세스를 통해 의도를 파악합니다. 첫째, 명시적인 명령어를 즉시 인식하는 빠른 경로, 둘째, 자유 형식 메시지에 대한 LLM 기반 의도 추론(Claude 코드 활용), 마지막으로 의도가 불분명할 경우 명확화 질문을 통해 응답하는 clarify 후속 조치입니다. 이 “인식 → 추론 → 명확화” 단계는 전체 설계 철학의 축소판입니다. 일반적인 경우는 즉각적이고 결정론적으로 처리되며, 유연한 경우는 모델이 담당하고, 불확실한 경우는 결코 잘못된 추측으로 잘못된 조치를 취하지 않습니다.

Herald는 참여자 및 귀속성 관리도 지원합니다. 운영자 사용자 이름 환경 변수(HERALD_<CHANNEL>_OPERATOR_USERNAME)와 참여자/귀속성 계약은 created_by/assigned_to 필드와 알림 @태깅을 제어해 누가 무엇을 했는지, 누가 알림을 받았는지 명확히 합니다. 거버넌스 측면에서 Herald는 Helix Constitution를 동일 위치 하위 모듈로 상속받아 규칙을 따르며, Docs Chain의 초기 프로덕션 사용자이기도 합니다. 전체 66개 마크다운 문서로 구성된 HTML/PDF/DOCX 코퍼스는 Docs Chain의 exec: 변환을 통해 연결되며, 오류 없이 검증됩니다. Herald는 주로 Shell/Go 도구로 구현되었으며, 계층화된 사양(V1→V2→V3→V4 버전 순차 적용)과 각 채널별 운영자 설정 가이드를 제공해 메신저 및 LLM/에이전트 디스패처를 지원합니다.

개발 배경

알림은 조용히 실패합니다. 잘못된 채널로 전송되거나 유실되거나, 사용자가 기억하지 못하는 엄격한 명령어 구문을 요구하죠. Herald는 안정적인 다중 채널 전송을 보장하고 자연스러운 언어로 응답할 수 있도록 설계되어, 알림이 신뢰할 수 있고 대응하기 쉬운 시스템이 되도록 합니다.

콘텐츠

왜 이것이 혁신적인 변화인가

보통 별도로 구매하는 두 가지 요소—신뢰할 수 있는 멀티채널 이벤트 라우팅과 자연어 인터페이스—를 하나로 통합한 시스템입니다. 운영자는 그저 말하기만 하면 소프트웨어가 그 의미를 파악합니다. 실제 운영 환경에서 신뢰성을 확보하는 핵심은 바로 명확성 확보 메커니즘입니다. 오인보다 질문을 우선하는 알림 시스템은 실제 운영에 적용할 수 있는 유일한 대안입니다.

혁신적인 요소

- 3단계 의도 처리 체계: 명령 빠른 경로 → LLM 추론 → 명확화 요청.
- 자연어 구독자 상호작용(명령어 구문 학습 불필요).
- 참여자 귀속 계약에 기반한 created_by/assigned_to + @태깅.
- 실제 Docs Chain 소비자(66개 문서 코퍼스, 멀티포맷, 검증 완료).

도전 과제와 해결 방안

- 모호한 자연어 의도: 세 단계 인식/추론/명확화 사다리를 통해 해결(맹목적 추측 배제).
- 안정적인 팬아웃: 수집 → 멀티채널 발송 설계로 알림이 정확한 목적지에 도달하도록 보장.
- 채널 간 정확한 귀속: 운영자 사용자명 환경 변수와 참여자 귀속 계약을 통해 해결.
- 문서 드리프트: Docs Chain를 통한 문서 코퍼스 검증 변환 파이프라인으로 해결.

기술 스택(이유와 방법)

- **Go** — 핵심 이벤트/디스패치 로직(조직별 언어 패턴 적용).
- **Shell** — 운영자 도구 및 설정 스크립트.
- **Claude 코드(LLM)** — 자유 형식 메시지에 대한 의도 추론 계층.
- 메신저 채널 어댑터 — 멀티채널 알림 팬아웃.
- **Docs Chain** — 문서 빌드/검증 파이프라인(Markdown→HTML/PDF/DOCX).
- **Helix Constitution** 서브모듈 — 상속된 거버넌스/규칙.